

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного
образовательного учреждения высшего
образования
**«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ
Г.В.ПЛЕХАНОВА»**

Техникум Пермского института (филиала)

Пояснительная записка

по теме: «Разработка WPF приложения для работы с партнерами
компаний»

по дисциплине «Технология разработки программного
обеспечения»

Работу выполнила
студентка группы ИПо-42
Будусова Т.М.

Пермь, 2026

СОДЕРЖАНИЕ

Введение	3
Раздел 1. Разработка базы данных	5
Раздел 2. Разработка программы	9
2.1. Создание библиотеки классов	9
2.2. Создание wrf приложения.....	13
2.3. Реализация логики работы программы.....	15
2.4. Модульное тестирование.....	21
2.5. Демонстрация работы программы	25
Заключение	32

ВВЕДЕНИЕ

Данная работа посвящена разработке программного модуля для учета партнеров производственной компании «Мастер пол». Компания занимается производством напольных покрытий и реализует свою продукцию через партнеров, которые обеспечивают доставку товаров до конечных потребителей. Для эффективного взаимодействия с партнерами и контроля их деятельности требуется система, позволяющая хранить и обрабатывать информацию о партнерах и истории реализации продукции в цифровом формате.

Актуальность данной работы обусловлена необходимостью автоматизации процессов учета и анализа данных о партнерах компании. Использование специализированного программного обеспечения позволяет значительно упростить процесс хранения информации, ускорить обработку данных и повысить эффективность управления партнерскими отношениями.

Целью работы является разработка WPF-приложения для учета партнеров компании, обеспечивающего хранение и обработку информации о партнерах, а также просмотр истории реализации продукции.

В рамках работы необходимо реализовать подсистему, включающую следующие основные функции: просмотр списка партнеров, добавление и редактирование данных о партнерах, а также просмотр истории реализации продукции партнером.

Для хранения данных используется реляционная база данных, созданная в системе управления базами данных PostgreSQL. Для разработки программного модуля применяется язык программирования C# и среда разработки Microsoft Visual Studio 2022 с использованием технологии Windows Presentation Foundation (WPF).

Разработанное приложение должно обеспечивать удобный пользовательский интерфейс для работы с данными, корректную обработку пользовательского ввода и взаимодействие с базой данных для получения и обновления информации о партнерах и истории их продаж.

РАЗДЕЛ 1. РАЗРАБОТКА БАЗЫ ДАННЫХ

Для обеспечения работы подсистемы учета партнеров была разработана реляционная база данных. База данных предназначена для хранения информации о партнерах компании, типах партнеров, реализуемой продукции и истории продаж. Использование базы данных позволяет централизованно хранить информацию, обеспечивать ее целостность и выполнять быстрый доступ к данным.

В качестве системы управления базами данных была выбрана PostgreSQL. Данная СУБД является одной из наиболее распространенных реляционных систем управления базами данных с открытым исходным кодом. PostgreSQL обеспечивает надежное хранение данных, поддерживает механизм внешних ключей для обеспечения ссылочной целостности и позволяет эффективно работать с большими объемами информации.

Создание базы данных выполнялось с использованием консольной утилиты `psql`, входящей в состав PostgreSQL. На первом этапе была создана роль пользователя для работы с базой данных:

```
CREATE ROLE app LOGIN PASSWORD '123456789';
```

Далее была создана база данных с учетом фамилии студента в английской транскрипции (по заданию):

```
CREATE DATABASE budusova_db_partners OWNER app;
```

После создания базы данных выполнено подключение к ней:

```
\c budusova_db_partners
```

Для удобства организации объектов в базе данных была создана отдельная схема `app` и установлен соответствующий `search_path`:

```
CREATE SCHEMA IF NOT EXISTS app AUTHORIZATION app;
```

```
ALTER ROLE app SET search_path TO app;
```

```
SET search_path TO app;
```

Далее была разработана структура базы данных, включающая таблицы, необходимые для хранения информации, используемой в подсистеме учета партнеров. При проектировании структуры базы данных была обеспечена третья нормальная форма (3НФ), что позволило исключить избыточность

данных и обеспечить логическую целостность информации.

В базе данных были созданы следующие основные таблицы:

- PartnerTypes — хранит информацию о типах партнеров;
- Partners — хранит информацию о партнерах компании;
- Products — хранит информацию о продукции компании;
- PartnerSales — хранит историю реализации продукции

партнерами.

Таблица PartnerTypes предназначена для хранения типов партнеров. Каждому типу присваивается уникальный идентификатор, который используется для связи с таблицей партнеров.

```
CREATE TABLE PartnerTypes (  
  Id SERIAL PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL  
);
```

Таблица Partners хранит основную информацию о партнерах компании. Для каждого партнера указывается наименование компании, юридический адрес, ФИО директора, контактные данные, рейтинг, а также ссылка на тип партнера.

```
CREATE TABLE Partners (  
  Id SERIAL PRIMARY KEY,  
  Name VARCHAR(200) NOT NULL,  
  LegalAddress TEXT,  
  DirectorFullName VARCHAR(200),  
  Phone VARCHAR(50),  
  Email VARCHAR(100),  
  Rating INTEGER,  
  PartnerTypeId INTEGER REFERENCES PartnerTypes(Id)  
);
```

Таблица Products предназначена для хранения информации о продукции компании, реализуемой через партнеров.

```
CREATE TABLE Products (  
  Id SERIAL PRIMARY KEY,
```

```
Name VARCHAR(200) NOT NULL,  
MinPartnerPrice NUMERIC  
);
```

Для хранения информации о реализации продукции партнерами была создана таблица PartnerSales. Данная таблица содержит данные о партнере, продукции, количестве проданного товара, цене и дате продажи.

```
CREATE TABLE PartnerSales (  
  Id SERIAL PRIMARY KEY,  
  PartnerId INTEGER REFERENCES Partners(Id),  
  ProductId INTEGER REFERENCES Products(Id),  
  Quantity INTEGER,  
  UnitPrice NUMERIC,  
  SaleDate DATE  
);
```

В данной таблице используются внешние ключи, обеспечивающие связь с таблицами партнеров и продукции. Это позволяет обеспечить ссылочную целостность данных и исключить появление некорректных записей в базе данных.

Разработанная структура базы данных соответствует требованиям третьей нормальной формы (3НФ). Каждая таблица хранит данные только об одной сущности, а связи между таблицами реализованы через внешние ключи. Такая организация обеспечивает целостность информации, исключает избыточность данных и позволяет при необходимости расширять систему без нарушения логики хранения информации.

На основе структуры базы данных была построена ER-диаграмма, отражающая связи между таблицами и их основными атрибутами (рис. 1.1). Диаграмма позволяет наглядно представить взаимосвязи между сущностями системы и служит наглядным инструментом для понимания архитектуры базы данных.

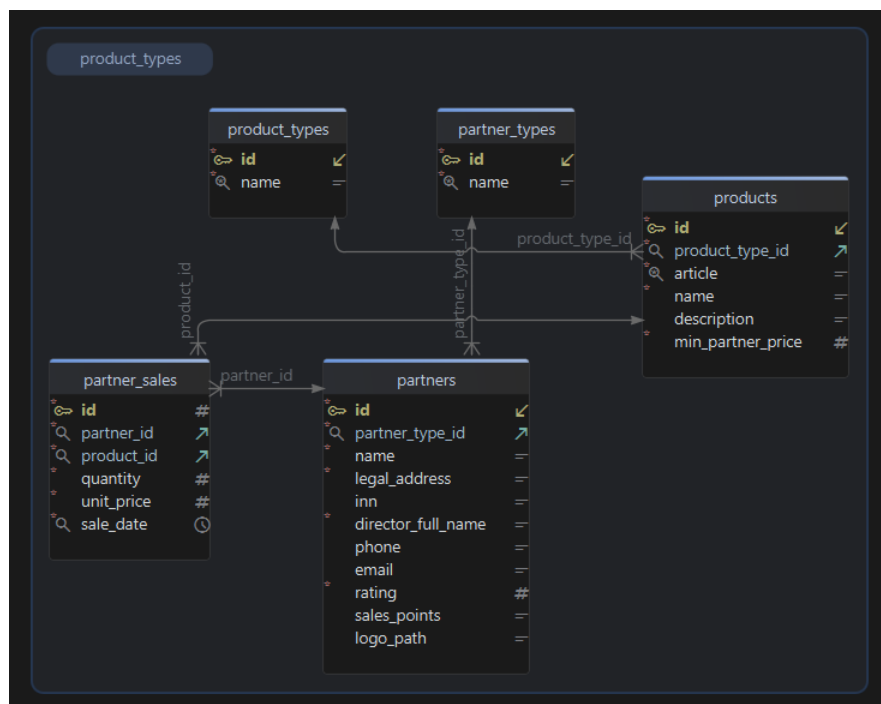


Рисунок 1.1 – ER-диаграмма базы данных

Разработанная база данных используется в программном модуле для хранения и обработки информации о партнерах компании, типах партнеров, реализуемой продукции и истории продаж. Все таблицы расположены в схеме app (рис. 1.2), а для пользователей и разработчиков настроены соответствующие права доступа, обеспечивающие корректную работу с данными.

```

    budusova_db_partners=> \dt
    Список таблиц
  
```

Схема	Имя	Тип	Владелец
app	partner_sales	таблица	app
app	partner_types	таблица	app
app	partners	таблица	app
app	product_types	таблица	app
app	products	таблица	app

(5 строк)

Рисунок 1.2 – Созданные таблицы в psql

РАЗДЕЛ 2. РАЗРАБОТКА ПРОГРАММЫ

2.1. Создание библиотеки классов

Для реализации подсистемы учета партнеров была создана отдельная библиотека классов (.NET Framework) на языке C#. Цель создания библиотеки заключалась в организации централизованной логики работы с данными и обеспечении удобного взаимодействия между программным модулем и базой данных PostgreSQL. Библиотека содержит модели данных, бизнес-логику и сервис для работы с базой, что позволяет отделить функционал обработки данных от пользовательского интерфейса.

В библиотеке классов были определены модели данных, полностью соответствующие таблицам базы данных, с учетом всех столбцов и типов данных. Каждая модель содержит свойства для хранения информации, которая в базе данных хранится в отдельных столбцах, а также дополнительные свойства для удобного отображения информации в интерфейсе.

Основные модели включают:

- `PartnerType` — соответствует таблице `PartnerTypes`. Содержит свойства `Id` и `Name` для хранения идентификатора типа партнера и его наименования.
- `Partner` — соответствует таблице `Partners`. Содержит свойства `Id`, `PartnerTypeId`, `Name`, `LegalAddress`, `INN`, `DirectorFullName`, `Phone`, `Email`, `Rating`, `SalesPoints` и `LogoPath`. Каждое свойство отражает соответствующий столбец в базе данных, обеспечивая полное представление информации о партнере (рис. 2.1.1).

```

namespace budusova_partners.Lib.Models
{
    [Table("partners", Schema = "app")]
    Ссылка: 13
    public class Partner
    {
        [Column("id")]
        Ссылка: 9
        public int Id { get; set; }

        [Column("partner_type_id")]
        Ссылка: 8
        public int PartnerTypeId { get; set; }

        [Column("name")]
        Ссылка: 8
        public string Name { get; set; }

        [Column("legal_address")]
        Ссылка: 5
        public string LegalAddress { get; set; }

        [Column("director_full_name")]
        Ссылка: 8
        public string DirectorFullName { get; set; }

        [Column("phone")]
        Ссылка: 5
        public string Phone { get; set; }

        [Column("email")]
        Ссылка: 5
        public string Email { get; set; }

        [Column("rating")]
        Ссылка: 10
        public int Rating { get; set; }

        Ссылка: 1
        public PartnerType PartnerType { get; set; }
        Ссылка: 3
        public ICollection<PartnerSale> PartnerSales { get; set; }

        [NotMapped]
        Ссылка: 1
        public string DiscountText { get; set; } // Для отображения скидки
    }
}

```

Рисунок 2.1.1 – Класс Partner

- ProductType — соответствует таблице ProductTypes. Содержит свойства Id и Name для хранения идентификатора и наименования типа продукции.
- Product — соответствует таблице Products. Содержит свойства Id, ProductTypeId, Article, Name, Description и MinPartnerPrice, отражающие все столбцы таблицы, включая тип продукции, артикул и минимальную цену для партнера (рис. 2.1.2).

```

namespace budusova_partners.Lib.Models
{
    [Table("products", Schema = "app")]
    Ссылка: 5
    public class Product
    {
        Ссылка: 4
        public int Id { get; set; }

        [Column("product_type_id")]
        Ссылка: 3
        public int ProductTypeId { get; set; }

        [Column("name")]
        Ссылка: 3
        public string Name { get; set; }

        [Column("min_partner_price")]
        Ссылка: 3
        public decimal MinPartnerPrice { get; set; }

        Ссылка: 0
        public List<PartnerSale> PartnerSales { get; set; }
    }
}

```

Рисунок 2.1.2 – Класс Product

- PartnerSale — соответствует таблице PartnerSales. Содержит свойства Id, PartnerId, ProductId, Quantity, UnitPrice и SaleDate, позволяя хранить информацию о каждой продаже, включая партнера, продукт, количество, цену и дату реализации.

Для работы с базой данных был создан контекст ApplicationDbContext, который обеспечивает связь между моделями и таблицами PostgreSQL через Entity Framework Core. Контекст содержит наборы данных (DbSet) для каждой сущности и реализует связи между таблицами: между партнером и его типом, между партнером и историей продаж, а также между продуктом и его типом. Это позволяет корректно загружать связанные данные при выполнении операций, таких как отображение истории продаж конкретного партнера.

В библиотеке реализован отдельный класс DiscountService, который отвечает за расчет индивидуальной скидки для партнера. Скидка определяется на основе суммарного объема продаж партнера за весь период работы и соответствует следующей шкале: до 10 000 — 0%, от 10 000 до 50 000 — 5%, от 50 000 до 300 000 — 10%, более 300 000 — 15%. Метод сервиса принимает общее количество реализованной продукции и возвращает соответствующий процент скидки (рис. 2.1.3). Кроме того, реализована проверка суммы продаж, которая не может быть отрицательной.

```

Ссылка: 1
public class DiscountService
{
    Ссылка: 1
    public static int CalculateDiscount(decimal totalSales)
    {
        if (totalSales < 0)
            throw new ArgumentException("Сумма продаж не может быть отрицательной");

        if (totalSales < 10000m) return 0;
        if (totalSales < 50000m) return 5;
        if (totalSales < 300000m) return 10;
        return 15;
    }

    Ссылка: 0
    public static string GetDiscountDescription(int discountPercent)
    {
        return discountPercent > 0 ? $"{discountPercent}%" : "нет";
    }
}

```

Рисунок 2.1.3 – Класс DiscountService

Структура библиотеки разработана с учетом принципов модульности и расширяемости. Каждый класс реализует отдельную сущность системы и содержит свойства для всех столбцов базы данных. Класс DiscountService инкапсулирует бизнес-логику расчета скидок, что позволяет легко изменять алгоритм без затрагивания модели данных и интерфейса.

Разработанная библиотека классов стала фундаментом подсистемы учета партнеров. Она обеспечивает хранение и обработку данных о партнерах, типах партнеров, продуктах и истории продаж, а также подготовку данных для интерфейса WPF и реализацию функции расчета скидок. Для библиотеки классов была создана диаграмма классов (рис. 2.1.4)

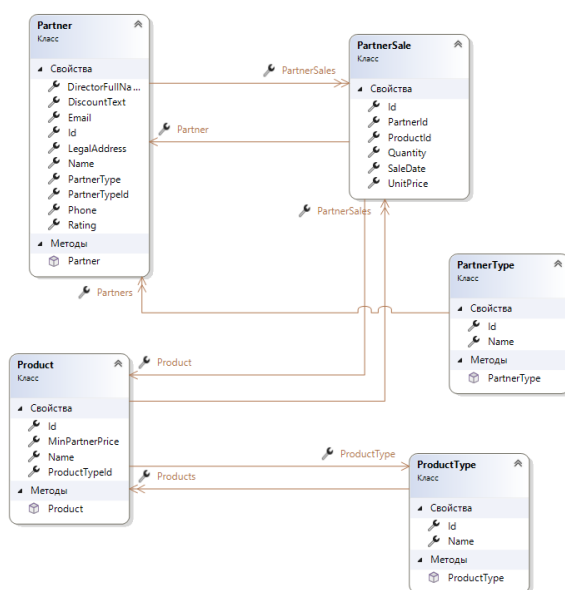


Рисунок 2.1.4 -Диаграмма классов

2.2. Создание WPF приложения

Для взаимодействия пользователя с подсистемой учета партнеров было разработано WPF-приложение с графическим интерфейсом на основе XAML.

Основная цель интерфейса — обеспечить доступ к функциям просмотра, добавления и редактирования информации о партнерах и отображения истории реализации продукции.

1. Главное окно программы (MainWindow)

Главное окно (рис. 2.2.1) состоит из двух основных областей, разделенных Grid с использованием ColumnDefinition:

1) Левая область — список партнеров (ListBox), позволяющий выбрать конкретного партнера для просмотра деталей.

Для каждого элемента списка используется DataTemplate, включающий Border и Grid.

Отображаются следующие данные:

- Тип партнера (PartnerType.Name)
- Наименование компании (Name)
- ФИО директора (DirectorFullName)
- Телефон (Phone)
- Рейтинг (Rating)
- Скидка (DiscountText)

Контекстное меню ContextMenu предоставляет возможность редактирования выбранного партнера через окно AddEditPartnerWindow.

2) Правая область — история продаж выбранного партнера (DataGrid).

Таблица содержит следующие столбцы: «Продукт» (Product.Name), «Количество» (Quantity), «Цена» (UnitPrice) и «Дата» (SaleDate).

История обновляется автоматически при выборе нового партнера в списке.

3) Разделитель (GridSplitter) расположен между областями и

позволяет изменять ширину панели с партнерами и панели с историей продаж.

Меню главного окна содержит:

- Пункт Файл, включающий команды:
 - «Добавить партнера» — открывает окно для добавления нового партнера.
 - «Выход» — закрывает приложение.
- Пункт Справка, содержащий команду «О программе» с информацией о приложении.

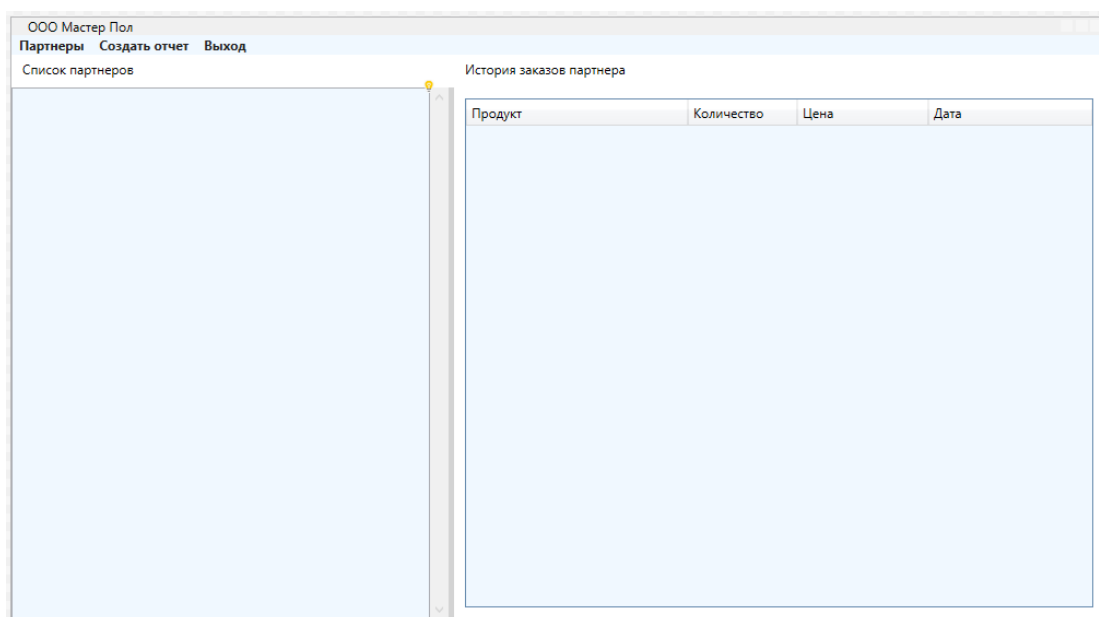


Рисунок 2.2.1 – Главное окно программы

2. Окно добавления и редактирования партнера (AddEditPartnerWindow)

Для ввода и редактирования данных о партнере используется окно AddEditPartnerWindow (рис. 2.2.2) на основе StackPanel с вертикальной ориентацией. Все поля выстроены последовательно: каждый TextBlock с подписью поля располагается над соответствующим элементом ввода (TextBox или ComboBox).

Используемые поля:

- Название — TextBox для ввода наименования компании.
- Тип — ComboBox с возможностью выбора типа партнера (PartnerType).

- Рейтинг — TextBox для ввода целого числа. Программа проверяет, что рейтинг неотрицательный.

- Адрес — TextBox для ввода юридического адреса компании.

- Директор — TextBox для ввода ФИО директора.

- Телефон — TextBox для ввода контактного номера.

- Email — TextBox для ввода электронной почты.

- В нижней части окна размещены кнопки:

Сохранить — сохраняет данные в базу данных и закрывает окно.

Отмена — закрывает окно без сохранения изменений.

Для обеспечения корректного отображения всех элементов даже на малых экранах используется отступы через Margin и Padding для отдельных элементов, а также фиксированная ширина окна и поля.

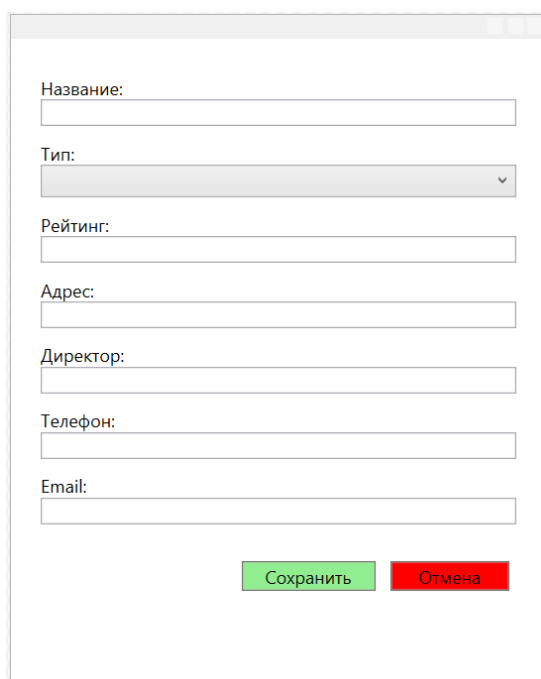
The image shows a Windows-style application window titled "Окно редактирования/добавления партнера". Inside the window, there are several labeled input fields: "Название:" (text box), "Тип:" (dropdown menu), "Рейтинг:" (text box), "Адрес:" (text box), "Директор:" (text box), "Телефон:" (text box), and "Email:" (text box). At the bottom center of the window, there are two buttons: a green "Сохранить" (Save) button and a red "Отмена" (Cancel) button.

Рисунок 2.2.2 – Окно редактирования/добавления партнера

2.3. Реализация логики работы программы

Логика работы разработанного WPF-приложения построена вокруг двух основных окон: главного окна (MainWindow) и окна добавления/редактирования партнера (AddEditPartnerWindow). Программа взаимодействует с базой данных PostgreSQL через Entity Framework Core, что

обеспечивает надежную загрузку, отображение, редактирование и сохранение данных о партнерах, типах партнеров, реализуемой продукции и истории продаж.

При запуске приложения выполняется метод `LoadPartners()`, который создает контекст базы данных (`AppDbContext`) для работы с таблицами `Partners`, `PartnerTypes` и `PartnerSales` (рис. 2.3.1). С помощью методов `Include` и `ThenInclude` загружается полный список партнеров вместе с их типами и историей продаж, обеспечивая готовность данных для отображения в интерфейсе. Для каждого партнера автоматически рассчитывается текущая скидка на основании суммы реализованной продукции за весь период работы. Этот расчет выполняется через отдельный сервис `DiscountService`, вызывающий метод `CalculateDiscount(totalSales)`. Результат сохраняется в поле `DiscountText` объекта партнера и отображается в правой части списка, предоставляя пользователю наглядную информацию о персональной скидке для каждого партнера. После загрузки данные связываются с визуальными элементами интерфейса, а источник `ItemsSource` для `ListBox` главного окна обновляется, что позволяет пользователю сразу видеть актуальный список партнеров с их рейтингом и текущей скидкой.

Ссылка 3

```
private void LoadPartners()
{
    try
    {
        var partners = db.Partners
            .Include(p => p.PartnerType)
            .Include(p => p.PartnerSales)
            .ThenInclude(ps => ps.Product)
            .ToList();

        foreach (var partner in partners)
        {
            int totalSales = partner.PartnerSales.Sum(ps => ps.Quantity);
            int discount = DiscountService.CalculateDiscount(totalSales);
            partner.DiscountText = $"{discount}%";
        }

        PartnersList.ItemsSource = null;
        PartnersList.ItemsSource = partners;
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Ошибка загрузки партнеров",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рисунок 2.3.1 - Метод `LoadPartners()`

Выбор конкретного партнера в списке (`ListBox`) инициирует событие `SelectionChanged`, которое определяет выбранный объект `Partner` и передает

его историю продаж в элемент DataGrid (SalesGrid) для отображения (рис. 2.3.2).

```
Ссылка 1
private void PartnersList_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (PartnersList.SelectedItem is Partner selectedPartner)
    {
        SalesGrid.ItemsSource = selectedPartner.PartnerSales.ToList();
    }
    else
    {
        SalesGrid.ItemsSource = null;
    }
}
```

Рисунок 2.3.2 – Метод PartnersList_SelectionChanged

История продаж формируется автоматически и содержит наименование продукции, количество реализованных единиц, цену за единицу и дату продажи. Если пользователь снимает выбор, ItemsSource таблицы обнуляется, обеспечивая пустое отображение для визуальной чистоты. Такой механизм синхронизации интерфейса и данных позволяет пользователю получать полное представление о продажах конкретного партнера и анализировать эффективность его работы.

Добавление нового партнера выполняется через кнопку или пункт меню «Добавить партнера». В этом случае создается экземпляр окна AddEditPartnerWindow без передачи существующего объекта партнера, что сигнализирует о создании нового объекта.

В окне пользователь вводит данные о партнере, включая наименование компании, тип партнера, рейтинг, юридический адрес, ФИО директора, телефон и email.

В форме пользователь заполняет поля: наименование компании, тип партнера из выпадающего списка, рейтинг, юридический адрес, ФИО директора, телефон и email. При нажатии кнопки «Сохранить» (метод Save_Click, рис. 2.3.3) происходит проверка корректности введенных данных: обязательно выбирается тип партнера, рейтинг должен быть числом и неотрицательным, а текстовые поля не должны быть пустыми. После успешной проверки создается объект Partner, который добавляется в контекст базы данных методом db.Partners.Add(currentPartner), и изменения фиксируются с помощью db.SaveChanges(). После закрытия формы

автоматически обновляется список партнеров в главном окне через повторный вызов LoadPartners(), что обеспечивает отображение нового партнера без перезапуска программы.

```
Ссылка: 1
private void Save_Click(object sender, RoutedEventArgs e)
{
    if (TypeBox.SelectedItem == null)
    {
        MessageBox.Show("Выберите тип партнера");
        return;
    }

    if (!int.TryParse(RatingBox.Text, out int rating))
    {
        MessageBox.Show("Рейтинг должен быть числом");
        return;
    }

    if (rating < 0)
    {
        MessageBox.Show("Рейтинг не может быть отрицательным", "Ошибка ввода");
        return;
    }

    if (currentPartner == null)
    {
        currentPartner = new Partner();
        db.Partners.Add(currentPartner);
    }

    currentPartner.Name = NameBox.Text;
    currentPartner.LegalAddress = AddressBox.Text;
    currentPartner.DirectorFullName = DirectorBox.Text;
    currentPartner.Phone = PhoneBox.Text;
    currentPartner.Email = EmailBox.Text;
    currentPartner.Rating = rating;
    currentPartner.PartnerTypeId = (int)TypeBox.SelectedValue;
    db.SaveChanges();
    Close();
}
```

Рисунок 2.3.3 – Метод Save_Click

Редактирование существующего партнера (Метод EditPartners_Click, рис. 2.3.4) реализуется двумя способами: через контекстное меню (ПКМ) на элементе списка партнеров или через кнопку «Редактировать партнера» в верхнем меню программы. При попытке редактирования через меню без выбора партнера выводится сообщение: «Выберите партнера для редактирования».

После выбора партнера создается окно AddEditPartnerWindow, которому передается выбранный объект Partner. В конструкторе окна все поля формы автоматически заполняются текущими значениями партнера, включая наименование компании, тип партнера, рейтинг, юридический адрес, ФИО директора, телефон и email.

После внесения изменений и нажатия кнопки «Сохранить» новые значения перезаписываются в объекте Partner, и изменения фиксируются в базе данных с помощью метода db.SaveChanges(). Сразу после закрытия окна выполняется повторная загрузка списка партнеров через метод LoadPartners(), что обеспечивает мгновенное обновление информации в интерфейсе.

Ссылка 2

```
private void EditPartner_Click(object sender, RoutedEventArgs e)
{
    // Проверяем, выбран ли партнер
    if (PartnersList.SelectedItem == null)
    {
        MessageBox.Show("Выберите партнера для редактирования",
            "Внимание", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var selectedPartner = (Partner)PartnersList.SelectedItem;

    // Открытие окна редактирования
    var editWindow = new AddEditPartnerWindow(selectedPartner);
    editWindow.ShowDialog();

    // Обновление списка после редактирования
    LoadPartners();
}
```

Рисунок 2.3.4 – Метод EditPartners_Click

Расчет скидки реализован отдельно в сервисе DiscountService (рис. 2.1.3). Алгоритм работы сервиса основан на суммировании общего количества продукции, реализованной партнером за весь период, и присвоении процентной скидки согласно заданным правилам: до 10 000 рублей — 0%, от 10 000 до 50 000 — 5%, от 50 000 до 300 000 — 10%, более 300 000 рублей — 15%. Скидка отображается рядом с информацией о партнере и обновляется каждый раз при загрузке данных.

История продаж выводится в DataGrid с указанием всех необходимых столбцов: «Продукт», «Количество», «Цена» и «Дата». Данные автоматически подгружаются при выборе партнера и обновляются при изменении базы данных. Такой подход обеспечивает удобный визуальный контроль над продажами каждого партнера и позволяет проводить быстрый анализ эффективности работы.

Программа предусматривает обработку исключительных ситуаций: проверку корректности ввода данных при добавлении или редактировании партнера, уведомление пользователя о некорректных значениях (например, отсутствие выбора типа партнера или отрицательный рейтинг), а также

обработку ошибок при загрузке данных из базы, например при отсутствии соединения с PostgreSQL. Все сообщения выводятся через диалоговые окна с информативным текстом, что предотвращает внесение некорректных записей и помогает пользователю корректно взаимодействовать с приложением.

Дополнительно реализованы стандартные элементы управления приложением, такие как выход из программы через закрытие главного окна (Close()) и отображение информации о подсистеме учета партнеров. Вся логика работы программы построена с учетом обеспечения целостности данных, удобства пользователя и возможности расширения функционала в будущем, а также обеспечивает синхронизацию интерфейса с базой данных и корректное отображение всех актуальных сведений о партнерах, типах, реализуемой продукции и истории продаж.

Кнопка «Удалить» предназначена для удаления выбранной записи из базы данных, будь то информация о партнере, типе продукции или истории продаж. При нажатии на кнопку программа проверяет, что пользователь действительно выбрал объект для удаления, после чего отображается окно подтверждения действия, предотвращающее случайное удаление данных. После подтверждения запись удаляется из базы данных, а интерфейс автоматически обновляется, отражая актуальное состояние. При невозможности удаления (например, если запись связана с другими данными) выводится соответствующее уведомление. Кнопка обеспечивает удобное и безопасное управление данными без риска потери целостности информации.

Кнопка «Сделать отчет» позволяет пользователю формировать сводные отчеты по партнерам, продукции и продажам. При нажатии кнопки формируется отчет и выводится сообщение, что отчет сформирован и сохранен (рис. 2.3.5, 2.3.6).

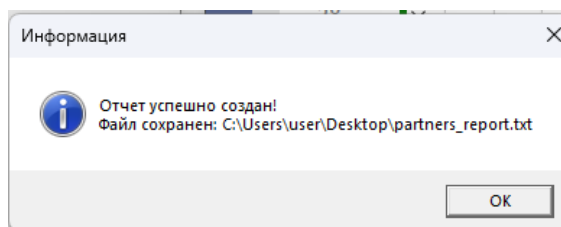


Рисунок 2.3.5 – Сообщение о сформированном отчете

```

ОТЧЕТ О ПАРТНЕРАХ И ПРОДАЖАХ
Дата формирования: 16.03.2026 0:00:45
-----
Партнер: Мир Плитки
Тип партнера: Розничный магазин
Директор: Иванов И.И.
Телефон: +7-900-111-22-33
Email: info@mirplitki.ru
Рейтинг: 9
История продаж:
  Товар: Ламинат Дуб Светлый | Количество: 120 | Цена: 1200,00 | Дата: 13.03.2026
  Товар: Паркет Классик | Количество: 50 | Цена: 3500,00 | Дата: 13.03.2026
-----
Партнер: КомфортПлитка
Тип партнера: Розничный магазин
Директор: Сидоров С.С.
Телефон: +7-902-333-44-55
Email: support@komfortplitka.ru
Рейтинг: 7
История продаж:
  Товар: Ламинат Орех Темный | Количество: 60 | Цена: 1450,00 | Дата: 13.03.2026
  Товар: Плитка ПВХ Standart | Количество: 100 | Цена: 950,00 | Дата: 13.03.2026
-----
Партнер: ПлиткаПро
Тип партнера: Дистрибьютор
Директор: Кузнецов К.К.
Телефон: +7-903-444-55-66
Email: sales@plitkapro.ru
Рейтинг: 10
История продаж:
  Товар: Ламинат Дуб Светлый | Количество: 320 | Цена: 1200,00 | Дата: 13.03.2026
  Товар: Ламинат Орех Темный | Количество: 150 | Цена: 2200,00 | Дата: 13.03.2026
-----
Партнер: ДомСтрой
Тип партнера: Розничный магазин
Директор: Федоров Ф.Ф.
Телефон: +7-904-555-66-77
Email: info@domstroy.ru
Рейтинг: 8
История продаж:
  Товар: Паркет Классик | Количество: 60 | Цена: 2500,00 | Дата: 13.03.2026
  Товар: Плитка ПВХ Standart | Количество: 90 | Цена: 310,00 | Дата: 13.03.2026
-----

```

Рисунок 2.3.6 – Сформированный отчет

2.4. Модульное тестирование

Для обеспечения надежности и корректной работы подсистемы учета партнеров были разработаны модульные тесты для библиотеки классов, в частности для сервиса расчета скидки (DiscountService) и взаимодействия с объектами данных (Partner, PartnerSales, Product, PartnerType). Основная цель тестов — проверить правильность вычислений, корректность обработки данных и устойчивость методов к некорректному вводу, а также убедиться в целостности данных при работе с коллекциями и базой данных.

Модульные тесты были реализованы с использованием фреймворка MSTest, обеспечивающего автоматическое тестирование методов C# и интеграцию тестов с процессом сборки проекта. Для организации тестов была

создана отдельная сборка `budusova_partners.Tests`, где расположены классы с тестовыми методами. Для изоляции тестов используется база данных `InMemoryDatabase`, позволяющая выполнять проверки без подключения к реальной PostgreSQL, что ускоряет тестирование и гарантирует независимость тестов друг от друга.

Перед запуском каждого теста выполняется метод `Setup()`, в котором:

- Создается контекст базы данных `_context` для работы с таблицами `Partners`, `PartnerTypes`, `Products` и `PartnerSales`.
- Очищаются все таблицы для обеспечения чистого состояния перед каждым тестом.
- Заполняются тестовые данные:
 - Типы партнеров: "Дистрибьютор", "Розничный магазин".
 - Партнеры: два тестовых партнера с разными типами и рейтингом.
 - Продукция: два тестовых продукта.
 - История продаж: несколько записей для первого партнера.
- Все изменения сохраняются через `_context.SaveChanges()`.

Такой подход позволяет проводить тестирование на предсказуемых данных и исключает влияние предыдущих тестов на текущий.

Основные направления тестирования

1. Тестирование расчета скидки партнера

Тесты проверяют корректность работы метода `DiscountService.CalculateDiscount(totalSales)` для различных диапазонов продаж:

- до 10 000 единиц — 0%;
- от 10 000 до 50 000 единиц — 5%;
- от 50 000 до 300 000 единиц — 10%;
- более 300 000 единиц — 15%.

Пример unit-теста:

```
[TestMethod]
public void Discount_ShouldBeCalculatedCorrectly(
```

```

{
    var discountService = new DiscountService();
    decimal discount = discountService.CalculateDiscount(15000); // Продажи = 15 000
    Assert.AreEqual(5, discount);
}

```

Каждый тест создает фиктивного партнера с заданным количеством проданной продукции и проверяет, что возвращаемое значение соответствует ожидаемому. Это гарантирует корректность логики расчета скидки для всех диапазонов и предотвращает нарушение работы подсистемы при изменении кода сервиса.

2. Тестирование корректности данных партнера

Проверяется, что объект `Partner` правильно сохраняет и возвращает данные:

- рейтинг корректно присваивается и не может быть отрицательным;
- партнер корректно связывается с типом через `PartnerTypeId`;
- поля `Name`, `LegalAddress`, `DirectorFullName`, `Phone` и `Email` сохраняются корректно.

Эти тесты обеспечивают целостность данных на уровне объекта и предотвращают появление некорректных записей в памяти приложения.

Пример unit-теста добавления партнера:

```

[TestMethod]
public void AddPartner_ShouldIncreaseCount
{
    int before = _context.Partners.Count();
    _context.Partners.Add(new Partner
    {
        PartnerTypeId = 2,
        Name = "Test Partner 3",
        DirectorFullName = "Сидоров С.С.",
        Rating = 7 });
}

```

```

_context.SaveChanges();
int after = _context.Partners.Count();
Assert.AreEqual(before + 1, after);}

```

3. Тестирование добавления и редактирования данных

Для проверки операций добавления и редактирования партнеров выполняются тесты, которые:

- проверяют увеличение количества партнеров при добавлении нового;
- проверяют корректное обновление полей существующего партнера;
- проверяют обработку ошибок, таких как отсутствие выбора типа партнера или попытка установить отрицательный рейтинг.

Данные тесты обеспечивают контроль целостности данных на уровне приложения до сохранения изменений в базе данных.

4. Тестирование работы с коллекциями и историей продаж

Для метода расчета общей суммы продаж и скидки создаются тесты с несколькими объектами PartnerSales:

- суммарное количество проданной продукции рассчитывается корректно;
- скидка определяется на основе актуальной суммы продаж;
- история продаж правильно связывается с партнером и корректно подгружается для отображения в интерфейсе программы.

Проверяется корректность расчетов и связей между объектами:

```

[TestMethod]
public void PartnerTotalSalesQuantity_ShouldReturnCorrectSum()
{
    var totalQuantity = _context.PartnerSale
        .Where(ps => ps.PartnerId == 1)
        .Sum(ps => ps.Quantity);
    Assert.AreEqual(70, totalQuantity)
}

```

}

5. Тестирование удаления данных

Проверяется удаление партнера и всех связанных с ним записей в PartnerSales.

- Удаление партнера автоматически удаляет все связанные продажи, что обеспечивает целостность данных;
- Это подтверждает правильность работы каскадного удаления и предотвращает появление "висящих" записей в базе.

6. Тестирование обновления данных

Проверяется, что изменения полей партнера, таких как рейтинг, корректно сохраняются в базе данных и отображаются при последующих загрузках.

Все тесты оформлены в виде отдельных методов с понятными названиями, отражающими тестируемый сценарий, и снабжены комментариями, поясняющими назначение проверки. Результаты тестирования представлены на рисунке 2.3.7.

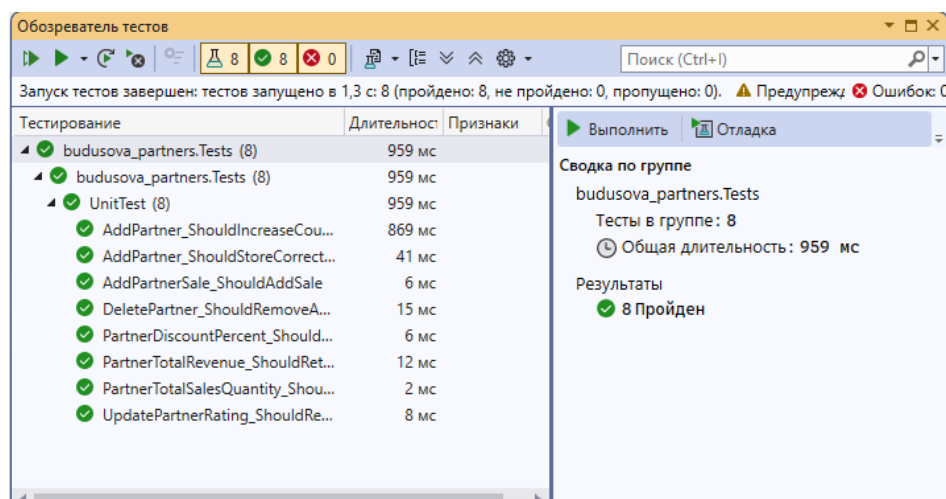


Рисунок 2.3.7 – Результаты тестирования

2.5. Демонстрация работы программы

Демонстрация работы программы позволяет наглядно продемонстрировать функциональность подсистемы учета партнеров компании «Мастер пол». При запуске приложения открывается главное окно (MainWindow), где отображается список всех партнеров. Для каждого

партнера показываются наименование компании, тип партнера, ФИО директора, контактные данные, рейтинг и текущая скидка, рассчитанная на основе объема продаж с использованием сервиса DiscountService (рис. 2.4.1).

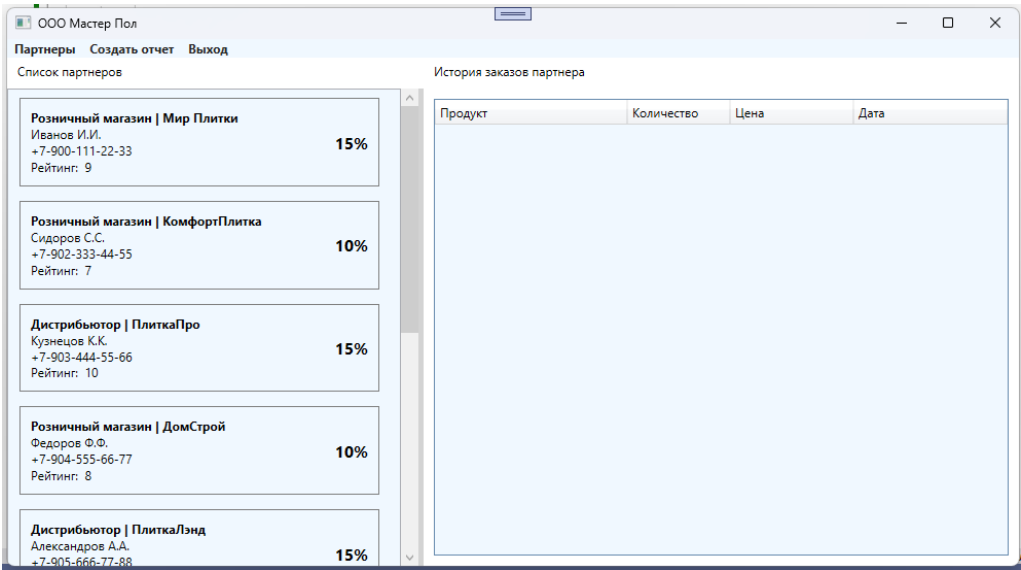


Рисунок 2.4.1 - Главное окно программы

Выбор конкретного партнера в списке автоматически загружает его историю продаж в таблицу DataGrid, где отображаются наименование продукции, количество проданных единиц, цена и дата продажи. Это позволяет быстро анализировать деятельность партнера и отслеживать динамику реализации продукции (рис. 2.4.2).

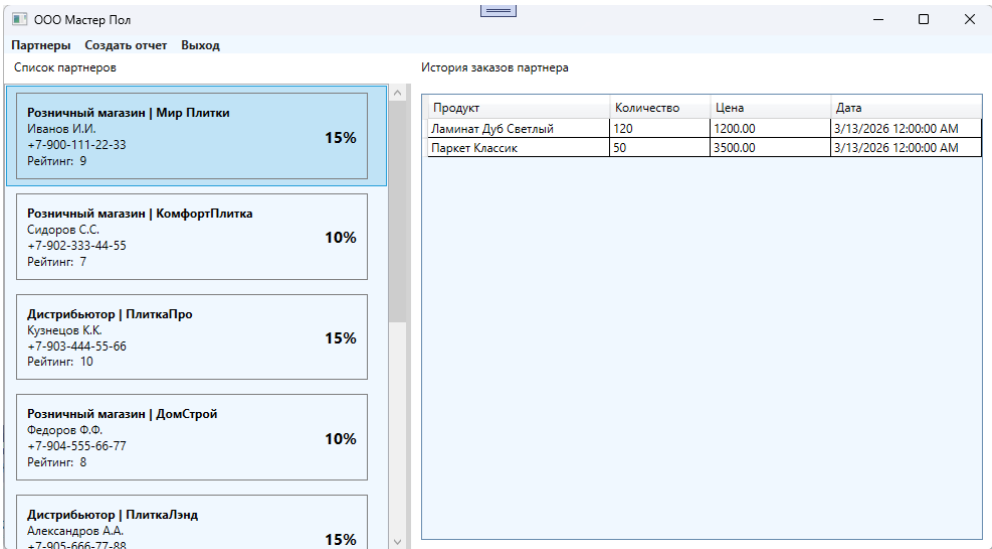


Рисунок 2.4.2 - Просмотр данных конкретного партнера

Добавление нового партнера осуществляется через кнопку «Добавить партнера» в верхнем меню (Партнеры/Добавить партнера) (рис. 2.4.3, 2.4.4).

Открывается окно AddEditPartnerWindow, где пользователь заполняет поля наименование компании, тип партнера из выпадающего списка, рейтинг, юридический адрес, ФИО директора, телефон и email. При нажатии кнопки «Сохранить» происходит проверка корректности введенных данных (выбран тип партнера, рейтинг — неотрицательное число, текстовые поля не пустые), после чего создается объект Partner, добавляется в контекст базы данных и сохраняется методом db.SaveChanges(). Список партнеров на главной форме обновляется автоматически после закрытия окна.

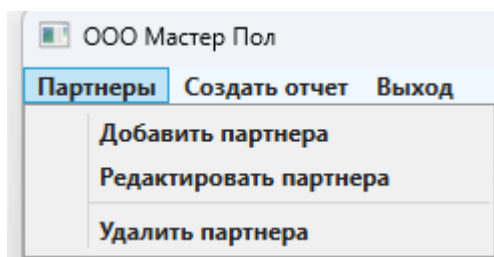


Рисунок 2.4.3 – Выбор действия в меню

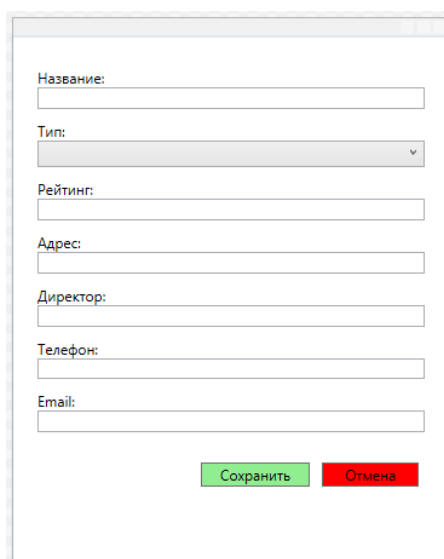


Рисунок 2.4.4 - Окно добавления партнера

При вводе рейтинга в виде дробного числа, выводится сообщение «Рейтинг должен быть целым числом» (рис. 2.4.5).

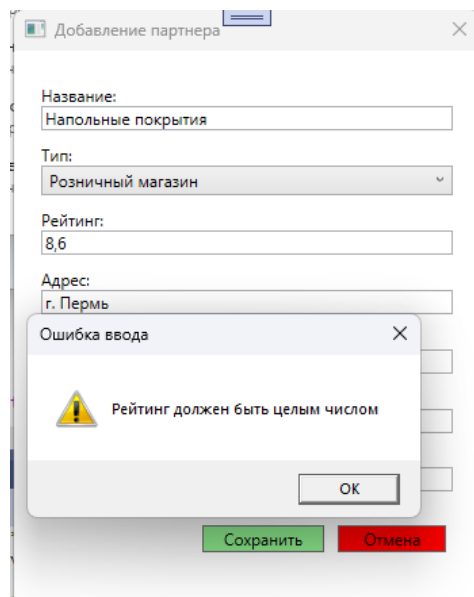


Рисунок 2.4.5 - Сообщение при вводе не целого числа

Поскольку в системе предусмотрен рейтинг от 0 до 10, то при вводе значения больше или меньше выйдет соответствующее сообщение (рис 2.4.6).

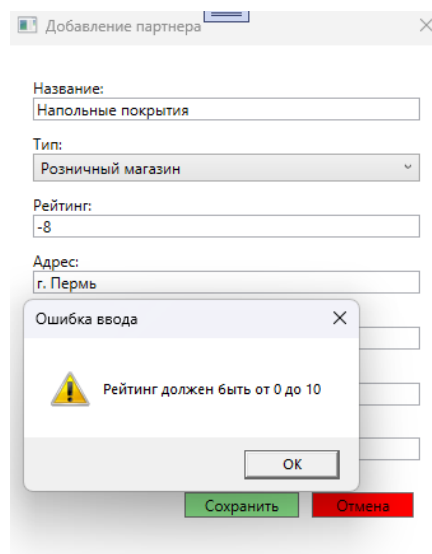


Рисунок 2.4.6 - Сообщение при вводе некорректного рейтинга

Так же, если не выбран тип партнера, пользователю выйдет сообщение «Выберите тип партнера» (рис. 2.4.7).

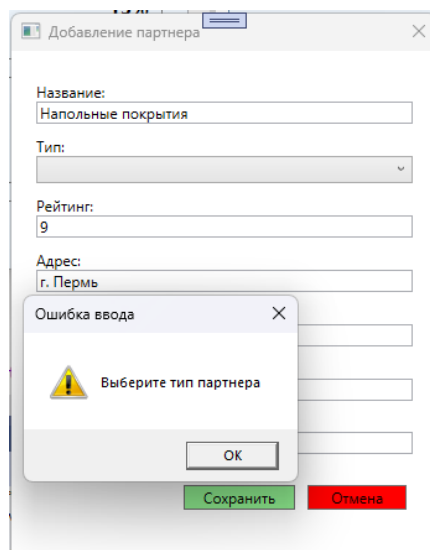


Рисунок 2.4.7 - Сообщение при отсутствии выбора типа партнера

Редактирование существующего партнера возможно двумя способами: через контекстное меню (ПКМ) на элементе списка или через кнопку «Редактировать партнера» в верхнем меню (рис. 2.4.8, 2.4.9). Если попытка редактирования через меню выполняется без выбора партнера, выводится сообщение: «Выберите партнера для редактирования» (рис. 2.4.10). После выбора партнера открывается окно AddEditPartnerWindow, где текущие значения всех полей заполняются автоматически. После внесения изменений и сохранения они фиксируются в базе данных, а список партнеров обновляется без необходимости перезапуска программы.

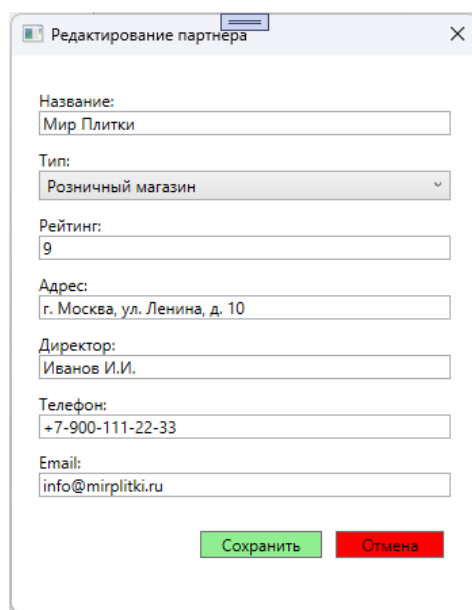


Рисунок 2.4.8 - Окно редактирования партнера

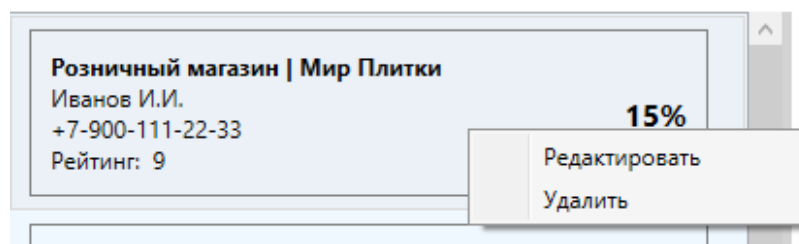


Рисунок 2.4.9 - Открытие окна редактирования с помощью ПКМ

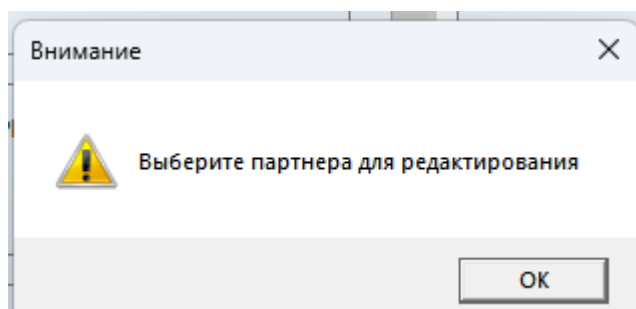


Рисунок 2.4.10 - Сообщение при нажатии кнопки редактировать без выбора

В подсистеме учета партнеров реализована возможность удаления партнера из базы данных. Удаление выполняется через меню списка партнеров на главной форме приложения и ПКМ. При нажатии кнопки удаления из меню без выбора партнера, выйдет сообщение «Сначала выберите партнера, которого необходимо удалить.», если партнер для удаления выбран, то удаление пройдет успешно. По аналогии с редактированием при нажатии правой кнопки мыши на выбранного партнера пользователю доступна команда «Удалить» (рис. 2.4.11).

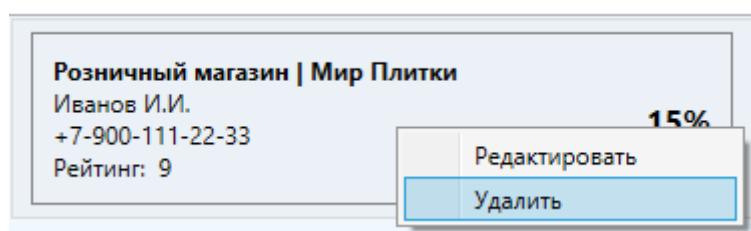


Рисунок 2.4.11 - Кнопка удалить

Перед выполнением удаления система выводит диалоговое окно подтверждения, предупреждающее пользователя о выполняемой операции. Это предотвращает случайное удаление данных. После подтверждения выбранный партнер удаляется из базы данных с использованием метода `db.Partners.Remove()`, а изменения сохраняются методом `db.SaveChanges()`.

После выполнения операции список партнеров автоматически

обновляется с помощью повторного вызова метода LoadPartners(), что позволяет сразу отобразить актуальные данные без перезапуска приложения.

Программа также предусматривает элементы управления интерфейсом: кнопка «Отмена» закрывает окно добавления или редактирования без сохранения изменений, меню «О программе» выводит информацию о подсистеме, а закрытие главного окна завершает работу приложения.

Демонстрация работы программы позволяет убедиться в корректной работе всех ключевых функций: загрузке и отображении данных, добавлении и редактировании партнеров через меню и ПКМ, просмотре истории продаж, автоматическом расчете скидки и стабильной работе приложения без ошибок.

ЗАКЛЮЧЕНИЕ

В рамках выполнения производственной практики был разработан программный модуль для учета партнеров производственной компании «Мастер пол». Приложение обеспечивает хранение и обработку информации о партнерах, их типах, реализуемой продукции и истории продаж, предоставляя удобный интерфейс для просмотра, добавления и редактирования данных о партнерах.

Для работы с данными была создана реляционная база данных в PostgreSQL с соблюдением третьей нормальной формы. Были реализованы таблицы PartnerTypes, Partners, Products и PartnerSales с настроенными связями через внешние ключи, что обеспечивает целостность данных и возможность их корректного обновления.

Программный модуль разработан на языке C# с использованием технологии WPF и Entity Framework Core, что позволяет взаимодействовать с базой данных через объектную модель. В приложении реализован сервис расчета скидки партнеров, который автоматически вычисляет процент скидки на основе объема продаж, а также отображает историю реализации продукции каждого партнера.

Для обеспечения надежности работы подсистемы были созданы модульные тесты, проверяющие корректность расчета скидок, сохранение и обновление данных о партнерах, работу с историей продаж и обработку коллекций данных. Тестирование гарантирует правильность работы критически важных функций приложения и стабильность при внесении изменений.

В результате работы создано полностью функционирующее WPF-приложение с базой данных PostgreSQL, обеспечивающее автоматизацию учета партнеров, удобство работы с данными и анализ продаж.

Все поставленные задачи выполнены, цель работы достигнута.